Parking Management - Rendu

1. Principe

Chaque étudiant est responsable d'un microservice.

Le document « Parking Management - Règles métier » définit le fonctionnement attendu du système.

Pendant les quatre jours de travail autonome, chacun doit faire progresser son service, travailler avec les autres étudiants lorsque des interfaces sont nécessaires et maintenir ses rendus à jour.

L'enseignant aide à cadrer et arbitre lorsque cela est nécessaire. Les choix d'interface entre services font partie du travail attendu.

---

2. Dossier de rendu

Chaque service possède un dossier dans l'espace partagé :

```
Parking/
Vehicles/
Pricing/
Payment/
Booking/
Messages/
Access/
```

Chaque dossier contient au minimum :

```
README.md
JOURNAL.md
```

Le code reste dans le dépôt GitHub ou GitLab correspondant.

---

3. README

Le README décrit l'état actuel du service.

Il doit contenir au minimum :

- rôle du service ;
- version actuelle ;
- URL du service déployé ;
- URL de la documentation Swagger/OpenAPI ;
- principales dépendances avec les autres services ;
- informations nécessaires pour tester le service.

Fonctionnalités

La liste des fonctionnalités doit être maintenue à jour.

Elle doit être écrite simplement, du point de vue fonctionnel.

Exemple pour Booking :

```
Fonctionnalités

[x] Je peux consulter les disponibilités d'un type de place.
[x] Je peux réserver une place sur un créneau.
[x] Je peux empêcher de réserver plus de places que la capacité disponible.
[x] Je peux demander le prix à Pricing.
[ ] Je peux faire payer la réservation avec Payment.
[ ] Access peut vérifier qu'un véhicule possède une réservation valable.
```

Une fonctionnalité n'est cochée que lorsqu'elle fonctionne réellement.

---

4. Journal de travail

Le fichier `JOURNAL.md` contient un compte rendu court à chaque journée de travail.

Exemple :

```
## 5 septembre

Fait
- réservation d'un créneau fonctionnelle ;
- service déployé sur Render ;
- contrat défini avec Pricing.

Problèmes
- paiement pas encore intégré.

Suite
- intégrer Payment ;
- tester le workflow complet.
```

L'objectif n'est pas de rédiger un rapport, mais de conserver une trace du travail et des difficultés rencontrées.

---

5. Aspects de maturité

Chaque service sera suivi selon plusieurs aspects indépendants.

Aspect	Attendu
Métier	Responsabilité, objets, règles et périmètre compris
API / Déploiement	Service en ligne, `/health`, Swagger, version identifiable
Intégration	Communications réelles avec les autres microservices
Interface	UI simple permettant d'utiliser ou d'observer le service
Qualité	Logs, erreurs propres, configuration, tests, éventuellement `/metrics`
CI/CD	Une modification validée sur GitHub/GitLab peut être automatiquement déployée
Sécurité	Les fonctions sensibles ne sont pas librement accessibles sur Internet

Chaque aspect peut être suivi avec quatre états :

```
0 - Non commencé
1 - En cours
2 - Fonctionnel
3 - Maîtrisé
```

Ces aspects sont indépendants : un service peut avoir une très bonne UI mais une intégration encore incomplète.

---

6. Authentification et droits

Dans un premier temps, les services peuvent être développés sans authentification afin de faciliter les échanges.

Avant le rendu final, les fonctions qui ne doivent pas être publiques doivent être protégées.

Une solution simple est suffisante pour l'exercice :

- API key ou token pour authentifier un appel ;
- ou JWT si l'étudiant souhaite aller plus loin ;
- secrets stockés dans les variables d'environnement et jamais dans Git ;
- distinction simple des droits lorsqu'elle est utile, par exemple `user` et `admin`.

Exemple :

```
Utilisateur
- consulter une disponibilité ;
- créer une réservation.

Administrateur
- modifier la capacité du parking ;
- modifier une grille tarifaire ;
- consulter des informations d'exploitation.
```

Il n'est pas demandé de développer un système complet de gestion des identités.

L'objectif est de montrer qu'un service exposé sur Internet peut contrôler qui a le droit d'effectuer une opération.

---

7. Suivi collectif

Un tableau partagé permet de suivre l'avancement :

Service	Métier	API	Intégration	UI	Qualité	CI/CD	Sécurité	Version	Dernière MAJ
Parking									
Vehicles									
Pricing									
Payment									
Booking									
Messages									
Access									

Ce tableau est visible par toute l'équipe.

Il sert à identifier rapidement l'état du système, les points à améliorer et les services qui doivent travailler ensemble.